

동시성 검증 리포트

좌석 예약 API의 동시 요청 처리 정합성 검증 (Redis SETNX 원자연산)

문서번호	CT-2026-0706-01	작성일	2026-07-06
대상 시스템	was-01(172.16.60.145), was-02(172.16.60.151), redis(172.16.60.144), db-01(172.16.60.143)	버전	1.0

1. 개요

다수의 사용자가 동일한 좌석을 동시에 선택·예약할 경우, 서버가 두 건 이상의 요청을 모두 성공 처리하면 중복판매(오버부킹)가 발생한다. 본 리포트는 좌석 예약 API가 Redis SETNX 원자연산을 통해 이러한 동시성 문제를 구조적으로 차단하는지 세 가지 조건에서 검증한 결과를 기록한다.

2. 대상 로직

좌석 예약 요청이 들어오면 WAS는 Redis에 seat:{gameId}:{seatId} 키를 NX(Not eXists) 옵션으로 설정을 시도한다. 이 명령은 Redis의 단일 스레드 특성상 원자적으로 처리되므로, 동시에 도달한 요청 중 정확히 하나만 키 생성에 성공한다.

```
app.post('/api/book-seat', async (req, res) => {
  const { gameId, seatId, buyerName } = req.body;
  const lockKey = `seat:${gameId}:${seatId}`;

  const acquired = await redisClient.set(lockKey, buyerName, { NX: true });
  if (!acquired) {
    return res.status(409).json({ message: '이미 선택된 좌석입니다' });
  }

  await db.query(
    'INSERT INTO bookings (game_id, seat_id, buyer_name) VALUES (?, ?, ?)',
    [gameId, seatId, buyerName]
  );
  res.status(200).json({ message: '예약 성공', gameId, seatId, buyerName });
});
```

DB의 bookings 테이블에는 UNIQUE KEY(game_id, seat_id) 제약을 추가로 걸어, 애플리케이션 로직에 결함이 있더라도 DB 레벨에서 중복 삽입을 2차로 차단하도록 설계했다.

3. 검증 방법

순번	테스트 조건	방식	목적
1	순차 curl 2회	동일 좌석(A5)에 curl로 순차 POST 요청 2회 전송	기본 동작(중복 거부) 확인
2	k6 내부 실행 (VU 100)	was-01 서버 내부에서 localhost로 100건 동시 요청	단일 서버 내 동시성 확인
3	k6 외부 실행 (VU 100)	맥북에서 queue-gate를 경유해 네트워크를 통과한 100건 동시 요청	실제 분산 환경에서의 동시성 확인

4. 테스트 스크립트

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    same_seat_attack: {
      executor: 'shared-iterations',
      vus: 100,
      iterations: 100,
      maxDuration: '10s',
    },
  },
};

export default function () {
  const payload = JSON.stringify({ gameId: 1, seatId: 'A7', buyerName: `user-${__VU}` });
  const res = http.post('http://172.16.60.145:3000/api/book-seat', payload, {
    headers: { 'Content-Type': 'application/json' },
  });
};
```

```
check(res, { '200 또는 409 응답': (r) => [200, 409].includes(r.status) });
}
```

조건 3(외부 실행)에서는 대상 URL을 `http://172.16.60.146/api/book-seat(queue-gate 경유)`로 변경하고 좌석 ID를 A7로 지정하여 실행했다.

5. 실행 결과

5.1 조건 1 — 순차 curl 2회 (좌석 A5)

```
$ curl -X POST http://localhost:3000/api/book-seat \
-H "Content-Type: application/json" \
-d '{"gameId":1,"seatId":"A5","buyerName":"테스트유저"}'
{"message":"예약 성공","gameId":1,"seatId":"A5","buyerName":"테스트유저"}

$ curl -X POST http://localhost:3000/api/book-seat \
-H "Content-Type: application/json" \
-d '{"gameId":1,"seatId":"A5","buyerName":"테스트유저2"}'
{"message":"이미 선택된 좌석입니다"}
```

5.2 조건 2 — k6 내부 실행 (좌석 A6)

```
checks.....: 100.00% 100 out of 100
http_req_duration.....: avg=30.6ms min=23.41ms med=30.84ms max=40.77ms p(95)=39.11ms
http_reqs.....: 100 1806.455655/s
running (00.1s), 000/100 VUs, 100 complete and 0 interrupted iterations

$ mysql -h 172.16.60.143 -u was_writer -p ticket \
-e "SELECT * FROM bookings WHERE seat_id='A6';"
+-----+-----+-----+-----+
| id | game_id | seat_id | buyer_name | created_at |
+-----+-----+-----+-----+
| 2 | 1 | A6 | user-98 | 2026-07-06 19:49:37 |
+-----+-----+-----+-----+
```

5.3 조건 3 — k6 외부 실행, queue-gate 경유 (좌석 A7)

```
$ mysql -h 172.16.60.143 -u was_writer -p ticket \
-e "SELECT * FROM bookings WHERE seat_id='A7';"
+-----+-----+-----+-----+
| id | game_id | seat_id | buyer_name | created_at |
+-----+-----+-----+-----+
| 3 | 1 | A7 | user-99 | 2026-07-06 20:01:14 |
+-----+-----+-----+-----+
```

6. 판정 기준 및 결과

조건	판정 기준	판정	비고
1	2번째 요청이 409로 거부되어야 함	합격	-
2	100건 동시요청 중 DB에 1행만 존재해야 함	합격	동일 좌석 대상 100건 중 1건만 INSERT
3	네트워크를 경유한 100건 동시요청에서도 DB에 1행만 존재해야 함	합격	단일 서버 내부 실행과 동일한 결과 확인

3가지 조건 모두에서 중복예약이 발생하지 않았으며, 동시 요청 규모(100건)와 실행 위치(내부/외부)에 관계없이 Redis 원자연산이 일관되게 동작함을 확인했다.

7. 결론

좌석 예약 API의 동시성 제어 로직은 설계 의도대로 동작하며, DB 트랜잭션 락 대신 Redis 원자연산을 선택한 설계 결정이 실제 부하 상황에서도 유효함을 확인했다. 다만 본 테스트는 좌석 1석에 대한 경쟁 상황만을 검증한 것으로, 다수 좌석에 대한 동시 예약(예: 4석 묶음 예매) 시나리오는 별도 검증이 필요하며 후속 과제로 남긴다.